ASSIGNMENT 5

Introduction to Hash Cryptography

Cryptography and Network Security

Coverage: Hash functions, security properties, applications, SHA-1/SHA-2/SHA-3, digital signatures, certificates, file integrity, and secure communication.

Note: Model answers to all five questions in the provided assignment, including examples, diagrams and comparison tables.

Q1. Define Hash Cryptography. Explain the Fundamental Principles and Techniques

Definition: Hash cryptography uses a cryptographic hash function to transform input data of practically any length into a fixed-length output called a hash value, digest, or message digest. The same input produces the same digest, but a secure hash is designed to make it infeasible to recover an input from its digest.

Fundamental principles

- **Deterministic:** identical input and algorithm produce an identical digest.
- **Fixed-length output:** SHA-256 produces a 256-bit digest regardless of the input length.
- **Preimage resistance:** given a digest, finding an input that produces it should be computationally infeasible.
- **Second-preimage resistance:** given one input, finding a different input with the same digest should be infeasible.
- **Collision resistance:** finding any two distinct inputs with the same digest should be infeasible.
- **Avalanche effect:** a tiny input change should alter many output bits in an unpredictable-looking way.

Techniques and working

Many hash functions process the message in blocks using repeated compression operations; SHA-3 instead uses the Keccak sponge construction. The precise internal steps depend on the algorithm.

Example: hash a file containing “Amount=100” using SHA-256. If its contents change to “Amount=1000”, the digest should change substantially. A receiver can calculate the digest of the received file and compare it with a trusted digest to detect changes. The expected digest must itself be authenticated.

Process diagram

Input	Hash algorithm	Output
Message/file: “Amount=100”	SHA-256	Fixed 256-bit digest

Important: hashing is not encryption. Encryption is designed to be reversible with a key; a cryptographic hash is not. A plain hash alone does not provide confidentiality or prove who created the data.

Q2. Role of Hash Functions in Cryptographic Applications

1. Data integrity verification

A sender or publisher provides a trusted digest for a file. The recipient hashes the received file and compares the result with the expected digest. A mismatch indicates a changed or different file. If the file and digest come from the same compromised source, an attacker may replace both; use a digital signature or trusted channel to authenticate the digest.

2. Digital signatures

Signature schemes commonly hash a message or include a hash-and-signature construction. The signer uses a private key and the verifier uses the corresponding public key. Hashing makes signing large documents practical and helps detect changes. A hash function alone does not create a digital signature.

3. Password storage

Services should not store plaintext passwords. Store password verifiers using a dedicated password-hashing scheme with a unique random salt and suitable work factor. Common choices include Argon2id, scrypt, bcrypt, or PBKDF2 where appropriate. Plain SHA-256 is too fast to use alone for password storage because guesses can be tested rapidly.

4. Message authentication

A Message Authentication Code (MAC) allows parties sharing a secret key to verify message integrity and authenticity. HMAC is a widely used construction that combines a cryptographic hash with a secret key. Unlike a plain hash, a correctly used HMAC cannot generally be forged without the key.

Application	Use of hash	Limitation
Integrity check	Compare calculated digest with trusted digest.	Digest must be authentic.
Digital signature	Hash is part of signing/verification.	Needs secure signature algorithm and key.
Password storage	Salted, slow password-hashing scheme.	Do not use plain fast SHA-256 alone.
Message authentication	HMAC combines hash and secret key.	Plain hash does not authenticate sender.

Q3. Advantages, Disadvantages and Security Properties

Advantages

- Produces a compact fixed-length digest for data of different sizes.
- Efficient for integrity checks and as a component in signatures and authentication protocols.
- Strong hashes make useful preimages and collisions computationally infeasible at the intended security level.
- Ordinary digest calculation does not require a secret key.

Disadvantages and limitations

- Hashing does not encrypt data or provide confidentiality.
- A plain hash does not establish identity; an attacker may replace both a file and its unauthenticated digest.
- Collisions necessarily exist because an unlimited set of inputs maps to a finite set of digests.
- Weak algorithms and incorrect use can undermine security; plain fast hashes are unsuitable for password storage.

Property	Meaning	Attacker goal
Preimage resistance	Given h, it is infeasible to find m such that H(m)=h.	Find an input matching a target digest.
Second-preimage resistance	Given m1, find m2 ≠ m1 with H(m2)=H(m1).	Replace a known message with another matching message.
Collision resistance	Find any m1 ≠ m2 such that H(m1)=H(m2).	Choose both inputs to collide.

Idealized work factors: for an ideal n-bit hash, generic preimage search takes about 2^n operations, while generic collision search takes about $2^{n/2}$ operations because of the birthday effect. Real weaknesses may reduce these costs.

Practical rule: use modern approved hashes such as SHA-256 or SHA-384 for general cryptographic hashing, and dedicated password-hashing algorithms for passwords.

Q4. Secure Hash Algorithm (SHA) Family: SHA-1, SHA-2 and SHA-3

The Secure Hash Algorithm (SHA) family is a set of cryptographic hash standards published by the U.S. National Institute of Standards and Technology (NIST).

Feature	SHA-1	SHA-2	SHA-3
Design	Iterative compression; 160-bit digest.	Iterative compression; includes SHA-224/256/384/512.	Keccak sponge construction; includes SHA3-224/256/384/512.
Standardization	Introduced in the 1990s.	Standardized in 2001.	Standardized in 2015.
Security status	Practical collision attacks exist; avoid for new security uses.	Widely used and considered secure for standard uses when correctly implemented.	Modern alternative with a different internal design from SHA-2.
Typical uses	Legacy compatibility only.	TLS, signatures, certificates and integrity checking.	Applications requiring SHA-3 or design diversity; SHAKE functions offer variable-length output.

SHA-1: produces a 160-bit digest. Practical collision attacks mean it should not be used for new signatures or other collision-sensitive applications.

SHA-2: includes SHA-256 (256-bit output) and SHA-512 (512-bit output), among other variants. It is common in modern security protocols.

SHA-3: uses the Keccak sponge construction, unlike SHA-2's compression design. Related SHAKE128 and SHAKE256 are extendable-output functions that can generate variable-length output.

Note: SHA-2 and SHA-3 are families of variants. Do not judge security by digest length alone; algorithm design and known attacks matter.

Q5. Applications of SHA Algorithms in Protocols and Information Security

1. Digital certificates

A certificate binds an identity to a public key and is signed by a Certificate Authority. A certificate-signature scheme uses a hash as part of the signature process. Modern certificate ecosystems use approved algorithms; SHA-1 signatures have been deprecated for modern public-key infrastructure because of collision weaknesses.

2. Digital signatures

A signing algorithm processes a message or digest with the signer's private key; the recipient verifies using the public key. Hashing helps efficiently handle large documents and detect modifications, assuming a secure signature scheme and hash.

3. File integrity checking

A software publisher may provide a SHA-256 checksum for an installer. The user calculates the downloaded file's digest and compares it with the publisher's value. A match shows that the file corresponds to that digest, but the expected checksum must come from an authenticated source.

4. Secure communication

Hash functions are used in TLS and other protocols for handshake transcript hashing, key derivation, and authentication constructions. HMAC appears in some protocol designs, while modern authenticated-encryption modes may provide message authentication directly. In TLS, a transcript hash binds handshake messages together, and a key derivation function derives traffic keys. Exact algorithms depend on protocol version and configuration.

5. Other security applications

- Package managers and software distributors use hashes to detect corruption and identify content.
- Version-control systems may use hash-based identifiers or integrity mechanisms; security depends on the system and algorithm.
- Backups and digital forensics use digests to help detect whether files have changed.

Conclusion: SHA algorithms are important building blocks for integrity, signatures, certificates and secure protocols. They must be selected according to current standards and combined with authentication and key management where trust or identity verification is required.

References for further reading

1. NIST FIPS 180-4, *Secure Hash Standard (SHS)*.
2. NIST FIPS 202, *SHA-3 Standard*.
3. NIST SP 800-107 Rev. 1, *Recommendation for Applications Using Approved Hash Algorithms*.
4. RFC 2104, *HMAC: Keyed-Hashing for Message Authentication*.
5. RFC 9106, *Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications*.